This problem set is not going to be collected, although solutions will be posted before the quiz. In addition to solving them, I recommend writing up solutions as if you were going to submit them. Unlike a problem set collected for credit, though, you are free to discuss with classmates and course staff and get help.

I recommend you first attempt the problems on your own, then discuss with classmates. After that, you may want to consult course staff at help hours or on Ed Discussion. After solutions are posted, consult the solutions and compare to what you wrote, and then discuss the differences with your study group.

1. In the lecture, we discussed the "famous problem." We saw an algorithm that uses exactly $3n - 3$ questions to identify a celebrity.

   (a) Is it true or false that any correct algorithm to identify a celebrity in a group of size n must ask $\Omega(n)$ questions? Justify your response in a few sentences. You may assume I am only asking about deterministic algorithms if you would prefer.

   (b) Give an algorithm to identify a celebrity with at most $3n - 3 - \log n$ questions asked, or correctly determines that no celebrity exists in the input group given. Note that this is strictly fewer than the algorithm given in class, but is not asymptotically better. You may assume that n is a power of two for purposes of this problem. Explain how your algorithm uses the desired number of questions.

2. Suppose I have an array A of n integer elements. Given a value x, I wish to find out if there are indices i and j such that $A[i] + A[j] == x$.

   For example, my value x might be 57 and my array might be:

-5	30	2	15	96	42

   In which case, I would answer true.

   You are allowed to make comparisons between elements, but cannot otherwise use their exact values. For example, do not use anything like hashing or radix sort.

   (a) The following algorithm correctly solves this problem. What is the running time? Use Θ notation.

   ```
   for i ← 1 to n do
     for j ← i + 1 to n do
       if A[i] + A[j] == x then
         return true
   return false
   ```

   (b) Give an algorithm to solve this problem in time $\mathcal{O}(n \log n)$. You may allocate $\mathcal{O}(1)$ additional memory in doing so.

   (c) Suppose that A is already sorted. Give an algorithm to solve this in time $\mathcal{O}(n)$.

3. Consider the problem of finding a *local minimum* in an array. The value $A[i]$ is a local minimum if - and only if - $A[i-1] \geq A[i] \leq A[i+1]$. Give an efficient **divide and conquer**

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

algorithm to find a local minimum in an array A in which $A[1] = A[n] = \infty$. State the runtime of your algorithm, both as a recurrence relation and in asymptotic notation and briefly (1-2 sentences is sufficient) explain the correctness of your algorithm.

For example, if your input array A is:

∞	10	23	17	20	1	2	3	4	-3	5	∞

then any of the grayed cells could be returned as a valid answer (you need only to find *one* local minimum, not all of them).

A solution with running time $\Omega(n)$ will receive no credit for this problem.

4. The Sorting Hats problem is as follows. I have a collection of n hats and n mannequins (statues that resemble a human and are used to model clothing and accessories). My goal is to put one hat on each mannequin head. Each hat is a distinct size, and no two heads are the same size. For each hat, there is exactly one head that the hat fits perfectly. I cannot directly compare two hats or two heads, nor can I inspect a single hat or a single head to determine its size.

   I can, however, try to place a hat on a head. If I do so, I find out one of the following pieces of information:

   - The hat fits the head perfectly
   - The hat is too small for the head
   - The hat is too big for the head

   We can then remove the hat from the head if we so choose.

   The goal of the Sorting Hat problem is to match each head with the hat that fits it perfectly while placing as few hats on heads as possible. Give an approach to do so. If there are n hats and n heads, express the number of times your approach will place a hat on a head in $\mathcal{O}$-notation and justify why that many such operations will happen. Clearly indicate if you are measuring *worst-case* or *expected case* (either is okay, but be sure to clearly indicate it).

   Good credit is available for any reasonably-efficient solution, clearly described, with more credit for more efficient solutions.

5. We've all heard of square root, but what about rectangle root? I define the k-rectangle root r of n to be:

$$(r)(r+k) = n$$

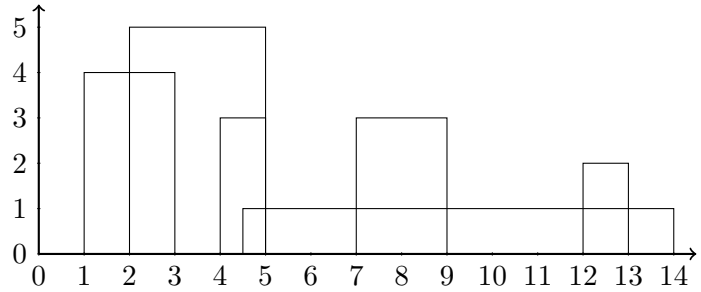
   For example, the 3-rectangle root of 130 is 10 because $10(10+3) = 130$.

   Write or describe an $\mathcal{O}(\log n)$ time algorithm to find the $\lfloor r \rfloor$, the floored k-rectangle root of n, given that $\lfloor r \rfloor, k, n$ are all natural numbers. Do not use a square root operation as part of your solution.

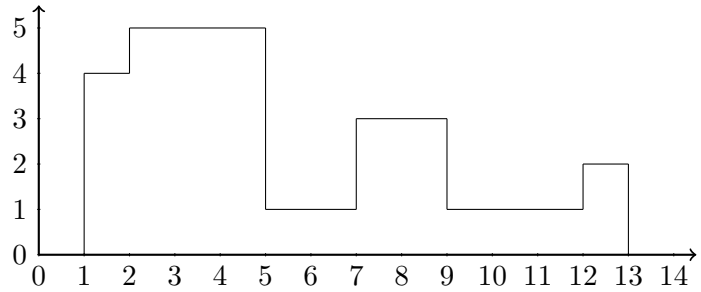
6. (2 points) Suppose you are given a description of rectangular, overlapping buildings and you would like to draw the corresponding skyline with no overlapping lines. A *skyline* is a list of tuples, each consisting of alternating x coordinates and the heights connecting them.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

The **input** is a series of 3-tuples; each tuple describes the left coordinate of the building, the height, and the right coordinate. For example, the input $\{(1,4,3),(2,5,5),(4,3,5),(4.5,1,14),(7,3,9),(12,2,$ would represent the following buildings:



You may assume that each building's left-coordinate is strictly before its right-coordinate, that there are no gaps in the buildings, and that each height value is positive. Give an efficient **divide and conquer** algorithm that, given the description of n buildings, forms the corresponding skyline; that is, the overlapping lines are removed. In the case of the above input, the result would be the output (1, 0), (1, 4), (2, 4), (2, 5), (5, 5), (5, 1), (7, 1), (7, 3), (9, 3), (9, 1), (12, 1), (12, 2), (13, 2), (13, 0), which visualized as:



Give also the running time of your algorithm, both as a recurrence relation and in closed form.

7. You are given a directed graph $G = (V, E)$ with exactly $\frac{n(n-1)}{2}$ edges. For every pair of nodes $u, v \in V$, one of the edges (u, v) and (v, u) are in the graph, and the other is not. You want to find a simple directed path which includes every node in the graph exactly once. It turns out that there is **always** such a path, which you will realize once you find the algorithm. Give an efficient Divide and Conquer algorithm to find such a path, and analyze the running time by setting up and solving a recurrence relation.

© CompSci 161 Spring 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.